JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Interface Set<E>

Type Parameters:

E - the type of elements maintained by this set

All Superinterfaces:

Collection<E>, Iterable<E>

All Known Subinterfaces:

EventSet, NavigableSet<E>, SequencedSet<E>, SortedSet<E>

All Known Implementing Classes:

AbstractSet, ConcurrentHashMap.KeySetView, ConcurrentSkipListSet, CopyOnWriteArraySet, EnumSet, HashSet, JobStateReasons, LinkedHashSet, TreeSet

---

```
public interface Set<E>
extends Collection<E>
```

A collection that contains no duplicate elements. More formally, sets contain no pair of elements `e1` and `e2` such that `e1.equals(e2)`, and at most one null element. As implied by its name, this interface models the mathematical *set* abstraction.

The `Set` interface places additional stipulations, beyond those inherited from the `Collection` interface, on the contracts of all constructors and on the contracts of the `add`, `equals` and `hashCode` methods. Declarations for other inherited methods are also included here for convenience. (The specifications accompanying these declarations have been tailored to the `Set` interface, but they do not contain any additional stipulations.)

The additional stipulation on constructors is, not surprisingly, that all constructors must create a set that contains no duplicate elements (as defined above).

Note: Great care must be exercised if mutable objects are used as set elements. The behavior of a set is not specified if the value of an object is changed in a manner that affects `equals` comparisons while the object is an element in the set. A special case of this prohibition is that it is not permissible for a set to contain itself as an element.

Some set implementations have restrictions on the elements that they may contain. For example, some implementations prohibit null elements, and some have restrictions on the types of their elements. Attempting to add an ineligible element throws an unchecked exception, typically `NullPointerException` or `ClassCastException`. Attempting to query the presence of an ineligible element may throw an exception, or it may simply return false; some

implementations will exhibit the former behavior and some will exhibit the latter. More generally, attempting an operation on an ineligible element whose completion would not result in the insertion of an ineligible element into the set may throw an exception or it may succeed, at the option of the implementation. Such exceptions are marked as "optional" in the specification for this interface.

Unmodifiable Sets

The `Set.of` and `Set.copyOf` static factory methods provide a convenient way to create unmodifiable sets. The `Set` instances created by these methods have the following characteristics:

- They are *unmodifiable*. Elements cannot be added or removed. Calling any mutator method on the Set will always cause `UnsupportedOperationException` to be thrown. However, if the contained elements are themselves mutable, this may cause the Set to behave inconsistently or its contents to appear to change.
- They disallow `null` elements. Attempts to create them with `null` elements result in `NullPointerException`.
- Unless otherwise specified, they are serializable if all elements are serializable.
- They reject duplicate elements at creation time. Duplicate elements passed to a static factory method result in `IllegalArgumentException`.
- The iteration order of set elements is unspecified and is subject to change.
- They are value-based. Programmers should treat instances that are equal as interchangeable and should not use them for synchronization, or unpredictable behavior may occur. For example, in a future release, synchronization may fail. Callers should make no assumptions about the identity of the returned instances. Factories are free to create new instances or reuse existing ones.
- They are serialized as specified on the Serialized Form page.

This interface is a member of the Java Collections Framework.

Since:

1.2

See Also:

`Collection`,
`List`,
`SortedSet`,
`HashSet`,
`TreeSet`,
`AbstractSet`,

Collections.singleton(java.lang.Object),
Collections.EMPTY_SET

Method Summary

All Methods | Static Methods

Modifier and Type	Method	Description
boolean	add(E e)	Adds the specified element to this set if it is not already present (optional operation).
boolean	addAll(Collection<? extends E> c)	Adds all of the elements in the specified collection to this set if they're not already present (optional operation).
void	clear()	Removes all of the elements from this set (optional operation).
boolean	contains(Object o)	Returns true if this set contains the specified element.
boolean	containsAll (Collection<?> c)	Returns true if this set contains all of the elements of the specified collection.
static <E> Set<E>	copyOf(Collection<? extends E> coll)	Returns an unmodifiable Set containing the elements of the given Collection.
boolean	equals(Object o)	Compares the specified object with this set for equality.
int	hashCode()	Returns the hash code value for this set.
boolean	isEmpty()	

		Returns `true` if this set contains no elements.
`Iterator<E>`	**`iterator`**`()`	Returns an iterator over the elements in this set.
`static <E> Set<E>`	**`of`**`()`	Returns an unmodifiable set containing zero elements.
`static <E> Set<E>`	**`of`**`(E e1)`	Returns an unmodifiable set containing one element.
`static <E> Set<E>`	**`of`**`(E... elements)`	Returns an unmodifiable set containing an arbitrary number of elements.
`static <E> Set<E>`	**`of`**`(E e1, E e2)`	Returns an unmodifiable set containing two elements.
`static <E> Set<E>`	**`of`**`(E e1, E e2, E e3)`	Returns an unmodifiable set containing three elements.
`static <E> Set<E>`	**`of`**`(E e1, E e2, E e3, E e4)`	Returns an unmodifiable set containing four elements.
`static <E> Set<E>`	**`of`**`(E e1, E e2, E e3, E e4, E e5)`	Returns an unmodifiable set containing five elements.
`static <E> Set<E>`	**`of`**`(E e1, E e2, E e3, E e4, E e5, E e6)`	Returns an unmodifiable set containing six elements.
`static <E> Set<E>`	**`of`**`(E e1, E e2, E e3, E e4, E e5, E e6, E e7)`	Returns an unmodifiable set containing seven elements.
`static <E> Set<E>`	**`of`**`(E e1, E e2, E e3, E e4, E e5, E e6, E e7, E e8)`	Returns an unmodifiable set containing eight elements.

static <E> Set<E>	**of**(E e1, E e2, E e3, E e4, E e5, E e6, E e7, E e8, E e9)	Returns an unmodifiable set containing nine elements.
static <E> Set<E>	**of**(E e1, E e2, E e3, E e4, E e5, E e6, E e7, E e8, E e9, E e10)	Returns an unmodifiable set containing ten elements.
static <E> Set<E>	**ofLazy**(Set<? extends E> elementCandidates, Predicate<? super E> computingFunction)	**Preview.** Returns a new lazily computed set whose logical membership for each distinct element candidate in the set of `elementCandidates` is computed via the provided `computingFunction` on demand.
boolean	**remove**(Object o)	Removes the specified element from this set if it is present (optional operation).
boolean	**removeAll**(Collection<?> c)	Removes from this set all of its elements that are contained in the specified collection (optional operation).
boolean	**retainAll**(Collection<?> c)	Retains only the elements in this set that are contained in the specified collection (optional operation).
int	**size**()	Returns the number of elements in this set (its cardinality).
default Spliterator<E>	**spliterator**()	Creates a `Spliterator` over the elements in this set.
Object[]	**toArray**()	

		Returns an array containing all of the elements in this set.
<T> T[]	**toArray**(T[] a)	Returns an array containing all of the elements in this set; the runtime type of the returned array is that of the specified array.

Methods declared in interface Collection

parallelStream, removeIf, stream, toArray

Methods declared in interface Iterable

forEach

Method Details

size

```
int size()
```

Returns the number of elements in this set (its cardinality). If this set contains more than `Integer.MAX_VALUE` elements, returns `Integer.MAX_VALUE`.

Specified by:

`size` in interface `Collection<E>`

Returns:

the number of elements in this set (its cardinality)

isEmpty

```
boolean isEmpty()
```

Returns `true` if this set contains no elements.

Specified by:

`isEmpty` in interface `Collection<E>`

Returns:

`true` if this set contains no elements

contains

`boolean contains(Object o)`

Returns `true` if this set contains the specified element. More formally, returns `true` if and only if this set contains an element `e` such that `Objects.equals(o, e)`.

Specified by:

`contains` in interface `Collection<E>`

Parameters:

`o` - element whose presence in this set is to be tested

Returns:

`true` if this set contains the specified element

Throws:

`ClassCastException` - if the type of the specified element is incompatible with this set (optional)

`NullPointerException` - if the specified element is null and this set does not permit null elements (optional)

iterator

`Iterator<E> iterator()`

Returns an iterator over the elements in this set. The elements are returned in no particular order (unless this set is an instance of some class that provides a guarantee).

Specified by:

`iterator` in interface `Collection<E>`

Specified by:

`iterator` in interface `Iterable<E>`

Returns:

an iterator over the elements in this set

toArray

`Object[] toArray()`

Returns an array containing all of the elements in this set. If this set makes any guarantees as to what order its elements are returned by its iterator, this method must return the elements in the same order.

The returned array will be "safe" in that no references to it are maintained by this set. (In other words, this method must allocate a new array even if this set
is backed by an array). The caller is thus free to modify the returned array.

This method acts as bridge between array-based and collection-based APIs.

Specified by:

`toArray` in interface `Collection<E>`

Returns:

an array containing all the elements in this set

toArray

`<T> T[] toArray(T[] a)`

Returns an array containing all of the elements in this set; the runtime type of the returned array is that of the specified array. If the set fits in the specified array, it is returned therein. Otherwise, a new array is allocated with the runtime type of the specified array and the size of this set.

If this set fits in the specified array with room to spare (i.e., the array has more elements than this set), the element in the array immediately following the end of the set is set to `null`. (This is useful in determining the length of this set *only* if the caller knows that this set does not contain any null elements.)

If this set makes any guarantees as to what order its elements are returned by its iterator, this method must return the elements in the same order.

Like the `toArray()` method, this method acts as bridge between array-based and collection-based APIs. Further, this method allows precise control over the runtime type of the output array, and may, under certain circumstances, be used to save allocation costs.

Suppose `x` is a set known to contain only strings. The following code can be used to dump the set into a newly allocated array of `String`:

```
    String[] y = x.toArray(new String[0]);
```

Note that `toArray(new Object[0])` is identical in function to `toArray()`.

Specified by:

`toArray` in interface `Collection<E>`

Type Parameters:

`T` - the component type of the array to contain the collection

Parameters:

`a` - the array into which the elements of this set are to be stored, if it is big enough; otherwise, a new array of the same runtime type is allocated for this purpose.

Returns:

an array containing all the elements in this set

Throws:

`ArrayStoreException` - if the runtime type of the specified array is not a supertype of the runtime type of every element in this set

`NullPointerException` - if the specified array is null

add

```
boolean add(E e)
```

Adds the specified element to this set if it is not already present (optional operation). More formally, adds the specified element `e` to this set if the set contains no element `e2` such that `Objects.equals(e, e2)`. If this set already contains the element, the call leaves the set unchanged and returns `false`. In combination with the restriction on constructors, this ensures that sets never contain duplicate elements.

The stipulation above does not imply that sets must accept all elements; sets may refuse to add any particular element, including `null`, and throw an exception, as described in the specification for `Collection.add`. Individual set implementations should clearly document any restrictions on the elements that they may contain.

Specified by:

`add` in interface `Collection`<`E`>

Parameters:

`e` - element to be added to this set

Returns:

`true` if this set did not already contain the specified element

Throws:

`UnsupportedOperationException` - if the `add` operation is not supported by this set

`ClassCastException` - if the class of the specified element prevents it from being added to this set

`NullPointerException` - if the specified element is null and this set does not permit null elements

`IllegalArgumentException` - if some property of the specified element prevents it from being added to this set

remove

```
boolean remove(Object o)
```

Removes the specified element from this set if it is present (optional operation). More formally, removes an element `e` such that `Objects.equals(o, e)`, if this set contains such an element. Returns `true` if this set contained the element (or equivalently, if this set changed as a result of the call). (This set will not contain the element once the call returns.)

Specified by:

`remove` in interface `Collection<E>`

Parameters:

`o` - object to be removed from this set, if present

Returns:

`true` if this set contained the specified element

Throws:

`ClassCastException` - if the type of the specified element is incompatible with this set (optional)

`NullPointerException` - if the specified element is null and this set does not permit null elements (optional)

`UnsupportedOperationException` - if the `remove` operation is not supported by this set

containsAll

```
boolean containsAll(Collection<?> c)
```

Returns `true` if this set contains all of the elements of the specified collection. If the specified collection is also a set, this method returns `true` if it is a *subset* of this set.

Specified by:

`containsAll` in interface `Collection<E>`

Parameters:

`c` - collection to be checked for containment in this set

Returns:

`true` if this set contains all of the elements of the specified collection

Throws:

`ClassCastException` - if the types of one or more elements in the specified collection are incompatible with this set (optional)

`NullPointerException` - if the specified collection contains one or more null elements and this set does not permit null elements (optional), or if the specified collection is null

See Also:

`contains(Object)`

addAll

`boolean addAll(Collection<? extends E> c)`

Adds all of the elements in the specified collection to this set if they're not already present (optional operation). If the specified collection is also a set, the `addAll` operation effectively modifies this set so that its value is the *union* of the two sets. The behavior of this operation is undefined if the specified collection is modified while the operation is in progress.

Specified by:

`addAll` in interface `Collection<E>`

Parameters:

`c` - collection containing elements to be added to this set

Returns:

`true` if this set changed as a result of the call

Throws:

`UnsupportedOperationException` - if the `addAll` operation is not supported by this set

`ClassCastException` - if the class of an element of the specified collection prevents it from being added to this set

`NullPointerException` - if the specified collection contains one or more null elements and this set does not permit null elements, or if the specified collection is null

`IllegalArgumentException` - if some property of an element of the specified collection prevents it from being added to this set

See Also:

`add(Object)`

retainAll

`boolean retainAll(Collection<?> c)`

Retains only the elements in this set that are contained in the specified collection (optional operation). In other words, removes from this set all of its elements that are not contained in the specified collection. If the specified collection is also a set, this operation effectively modifies this set so that its value is the *intersection* of the two sets.

Specified by:

`retainAll` in interface `Collection<E>`

Parameters:

`c` - collection containing elements to be retained in this set

Returns:

`true` if this set changed as a result of the call

Throws:

`UnsupportedOperationException` - if the `retainAll` operation is not supported by this set

`ClassCastException` - if the class of an element of this set is incompatible with the specified collection (optional)

`NullPointerException` - if this set contains a null element and the specified collection does not permit null elements (optional), or if the specified collection is null

See Also:

`remove(Object)`

removeAll

```
boolean removeAll(Collection<?> c)
```

Removes from this set all of its elements that are contained in the specified collection (optional operation). If the specified collection is also a set, this operation effectively modifies this set so that its value is the *asymmetric set difference* of the two sets.

Specified by:

`removeAll` in interface `Collection<E>`

Parameters:

`c` - collection containing elements to be removed from this set

Returns:

`true` if this set changed as a result of the call

Throws:

`UnsupportedOperationException` - if the `removeAll` operation is not supported by this set

`ClassCastException` - if the class of an element of this set is incompatible with the specified collection (optional)

`NullPointerException` - if this set contains a null element and the specified collection does not permit null elements (optional), or if the specified collection is null

See Also:

`remove(Object)`, `contains(Object)`

clear

```
void clear()
```

Removes all of the elements from this set (optional operation). The set will be empty after this call returns.

Specified by:

`clear` in interface `Collection<E>`

Throws:

`UnsupportedOperationException` - if the `clear` method is not supported by this set

equals

```
boolean equals(Object o)
```

Compares the specified object with this set for equality. Returns `true` if the specified object is also a set, the two sets have the same size, and every member of the specified set is contained in this set (or equivalently, every member of this set is contained in the specified set). This definition ensures that the equals method works properly across different implementations of the set interface.

Specified by:

`equals` in interface `Collection<E>`

Overrides:

`equals` in class `Object`

Parameters:

`o` - object to be compared for equality with this set

Returns:

`true` if the specified object is equal to this set

See Also:

`Object.equals(Object)`, `equals(Object)`, `List.equals(Object)`

hashCode

`int hashCode()`

Returns the hash code value for this set. The hash code of a set is defined to be the sum of the hash codes of the elements in the set, where the hash code of a `null` element is defined to be zero. This ensures that `s1.equals(s2)` implies that `s1.hashCode()==s2.hashCode()` for any two sets `s1` and `s2`, as required by the general contract of `Object.hashCode()`.

Specified by:

`hashCode` in interface `Collection<E>`

Overrides:

`hashCode` in class `Object`

Returns:

the hash code value for this set

See Also:

`Object.equals(Object)`, `equals(Object)`

spliterator

`default Spliterator<E> spliterator()`

Creates a `Spliterator` over the elements in this set.

The `Spliterator` reports `Spliterator.DISTINCT`. Implementations should document the reporting of additional characteristic values.

Specified by:

`spliterator` in interface `Collection<E>`

Specified by:

`spliterator` in interface `Iterable<E>`

Implementation Requirements:

The default implementation creates a *late-binding* spliterator from the set's `Iterator`. The spliterator inherits the *fail-fast* properties of the set's iterator.

The created `Spliterator` additionally reports `Spliterator.SIZED`.

Implementation Note:

The created `Spliterator` additionally reports `Spliterator.SUBSIZED`.

Returns:

a `Spliterator` over the elements in this set

Since:

1.8

of

```
static <E> Set<E> of()
```

Returns an unmodifiable set containing zero elements. See Unmodifiable Sets for details.

Type Parameters:

`E` - the `Set`'s element type

Returns:

an empty `Set`

Since:

9

of

```
static <E> Set<E> of(E e1)
```

Returns an unmodifiable set containing one element. See Unmodifiable Sets for details.

Type Parameters:

`E` - the `Set`'s element type

Parameters:

`e1` - the single element

Returns:

a `Set` containing the specified element

Throws:

`NullPointerException` - if the element is `null`

Since:

9

of

```
static <E> Set<E> of(E e1,
                     E e2)
```

Returns an unmodifiable set containing two elements. See Unmodifiable Sets for details.

Type Parameters:

`E` - the `Set`'s element type

Parameters:

e1 - the first element

e2 - the second element

Returns:

a Set containing the specified elements

Throws:

IllegalArgumentException - if the elements are duplicates

NullPointerException - if an element is null

Since:

9

of

```
static <E> Set<E> of(E e1,
                     E e2,
                     E e3)
```

Returns an unmodifiable set containing three elements. See Unmodifiable Sets for details.

Type Parameters:

E - the Set's element type

Parameters:

e1 - the first element

e2 - the second element

e3 - the third element

Returns:

a Set containing the specified elements

Throws:

IllegalArgumentException - if there are any duplicate elements

NullPointerException - if an element is null

Since:

9

of

```
static <E> Set<E> of(E e1,
                     E e2,
                     E e3,
                     E e4)
```

Returns an unmodifiable set containing four elements. See Unmodifiable Sets for details.

Type Parameters:

`E` - the `Set`'s element type

Parameters:

`e1` - the first element

`e2` - the second element

`e3` - the third element

`e4` - the fourth element

Returns:

a `Set` containing the specified elements

Throws:

`IllegalArgumentException` - if there are any duplicate elements

`NullPointerException` - if an element is `null`

Since:

9

of

```
static <E> Set<E> of(E e1,
                     E e2,
                     E e3,
                     E e4,
                     E e5)
```

Returns an unmodifiable set containing five elements. See Unmodifiable Sets for details.

Type Parameters:

`E` - the `Set`'s element type

Parameters:

`e1` - the first element

`e2` - the second element

`e3` - the third element

`e4` - the fourth element

`e5` - the fifth element

Returns:

a `Set` containing the specified elements

Throws:

`IllegalArgumentException` - if there are any duplicate elements

`NullPointerException` - if an element is `null`

Since:

9

of

```
static <E> Set<E> of(E e1,
                     E e2,
                     E e3,
                     E e4,
                     E e5,
                     E e6)
```

Returns an unmodifiable set containing six elements. See Unmodifiable Sets for details.

Type Parameters:

`E` - the `Set`'s element type

Parameters:

`e1` - the first element

`e2` - the second element

`e3` - the third element

`e4` - the fourth element

`e5` - the fifth element

`e6` - the sixth element

Returns:

a `Set` containing the specified elements

Throws:

`IllegalArgumentException` - if there are any duplicate elements

`NullPointerException` - if an element is `null`

Since:

9

of

```
static <E> Set<E> of(E e1,
                     E e2,
                     E e3,
                     E e4,
                     E e5,
                     E e6,
                     E e7)
```

Returns an unmodifiable set containing seven elements. See Unmodifiable Sets for details.

Type Parameters:

`E` - the `Set`'s element type

Parameters:

`e1` - the first element

`e2` - the second element

`e3` - the third element

`e4` - the fourth element

`e5` - the fifth element

`e6` - the sixth element

`e7` - the seventh element

Returns:

a `Set` containing the specified elements

Throws:

`IllegalArgumentException` - if there are any duplicate elements

`NullPointerException` - if an element is `null`

Since:

9

of

```
static <E> Set<E> of(E e1,
                     E e2,
                     E e3,
                     E e4,
                     E e5,
                     E e6,
                     E e7,
                     E e8)
```

Returns an unmodifiable set containing eight elements. See Unmodifiable Sets for details.

Type Parameters:

`E` - the `Set`'s element type

Parameters:

`e1` - the first element

`e2` - the second element

`e3` - the third element

`e4` - the fourth element

`e5` - the fifth element

`e6` - the sixth element

`e7` - the seventh element

`e8` - the eighth element

Returns:

a `Set` containing the specified elements

Throws:

`IllegalArgumentException` - if there are any duplicate elements

`NullPointerException` - if an element is `null`

Since:

9

of

```
static <E> Set<E> of(E e1,
                     E e2,
                     E e3,
                     E e4,
                     E e5,
                     E e6,
                     E e7,
                     E e8,
                     E e9)
```

Returns an unmodifiable set containing nine elements. See Unmodifiable Sets for details.

Type Parameters:

`E` - the `Set`'s element type

Parameters:

`e1` - the first element

e2 - the second element

e3 - the third element

e4 - the fourth element

e5 - the fifth element

e6 - the sixth element

e7 - the seventh element

e8 - the eighth element

e9 - the ninth element

Returns:

a `Set` containing the specified elements

Throws:

`IllegalArgumentException` - if there are any duplicate elements

`NullPointerException` - if an element is `null`

Since:

9

of

```
static <E> Set<E> of(E e1,
                     E e2,
                     E e3,
                     E e4,
                     E e5,
                     E e6,
                     E e7,
                     E e8,
                     E e9,
                     E e10)
```

Returns an unmodifiable set containing ten elements. See Unmodifiable Sets for details.

Type Parameters:

`E` - the `Set`'s element type

Parameters:

e1 - the first element

e2 - the second element

e3 - the third element

e4 - the fourth element

`e5` - the fifth element

`e6` - the sixth element

`e7` - the seventh element

`e8` - the eighth element

`e9` - the ninth element

`e10` - the tenth element

Returns:

a `Set` containing the specified elements

Throws:

`IllegalArgumentException` - if there are any duplicate elements

`NullPointerException` - if an element is `null`

Since:

9

of

```
@SafeVarargs
static <E> Set<E> of(E... elements)
```

Returns an unmodifiable set containing an arbitrary number of elements. See Unmodifiable Sets for details.

API Note:

This method also accepts a single array as an argument. The element type of the resulting set will be the component type of the array, and the size of the set will be equal to the length of the array. To create a set with a single element that is an array, do the following:

```
String[] array = ... ;
Set<String[]> list = Set.<String[]>of(array);
```

This will cause the `Set.of(E)` method to be invoked instead.

Type Parameters:

`E` - the `Set`'s element type

Parameters:

`elements` - the elements to be contained in the set

Returns:

a `Set` containing the specified elements

Throws:

`IllegalArgumentException` - if there are any duplicate elements

`NullPointerException` - if an element is `null` or if the array is `null`

Since:

9

copyOf

```
static <E> Set<E> copyOf(Collection<? extends E> coll)
```

Returns an unmodifiable Set containing the elements of the given Collection. The given Collection must not be null, and it must not contain any null elements. If the given Collection contains duplicate elements, an arbitrary element of the duplicates is preserved. If the given Collection is subsequently modified, the returned Set will not reflect such modifications.

Implementation Note:

If the given Collection is an unmodifiable Set, calling copyOf will generally not create a copy.

Type Parameters:

`E` - the `Set`'s element type

Parameters:

`coll` - a `Collection` from which elements are drawn, must be non-null

Returns:

a `Set` containing the elements of the given `Collection`

Throws:

`NullPointerException` - if coll is null, or if it contains any nulls

Since:

10

ofLazy

```
static <E> Set<E> ofLazy(Set<? extends E> elementCandidates,
                         Predicate<? super E> computingFunction)
```

> **`ofLazy` is a preview API of the Java platform.**
> *Programs can only use `ofLazy` when preview features are enabled. Preview features may be removed in a future release, or upgraded to permanent features of the Java platform.*

Returns a new lazily computed set whose logical membership for each distinct element candidate in the set of `elementCandidates` is computed via the provided `computingFunction` on demand.

In the following, the term *membership status* is used to indicate whether an element belongs to the returned set or not. That is, if the membership status for element `E` is `true`, then `E` *is* a member of the returned set. Conversely, if the membership status for element `E` is `false`, then `E` *is not* a member of the returned set.

The returned set is an unmodifiable set. The elements in the returned set are derived from the element candidates given at construction in combination with evaluating each element's membership status. The set's element membership statuses are lazily computed via the provided `computingFunction` when first accessed (e.g., via Set::contains). Once the membership status has been successfully computed for an element candidate, the associated membership status is initialized (i.e., either as *a logical member* or as *a logical non-member*).

The provided computing function is guaranteed to be invoked at most once per element candidate, even in a multi-threaded environment. Competing threads accessing an element candidate already under membership status computation will block until the membership status of the element candidate is computed or the computing function completes abnormally.

If evaluation of the provided computing function throws an unchecked exception (for an element candidate), the lazy membership status is not initialized but instead transitions to an error state whereafter a NoSuchElementException is thrown with the unchecked exception as a cause. Subsequent Set::contains calls for the same membership candidate throw NoSuchElementException (without ever invoking the computing function again) with no cause and with a message that includes the name of the original unchecked exception's class.

All failures are handled in this way. There is a special case that causes unchecked exceptions to be thrown:

If the computing function recursively invokes itself (for the same membership candidate) via the returned lazy set, a NoSuchElementException (with an IllegalStateException as a cause) will be thrown.

The returned set's Object methods; equals(), hashCode(), and toString() methods may trigger initialization of one or more lazy elements. If initialization fails for at least one element, the hashCode() and toString() methods throw NoSuchElementException, and the Object.equals(Object) throw NoSuchElementException if attempting to compare an element that could not be computed.

The returned lazy set strongly references its underlying computing function used to compute membership status at least as long as there are uncomputed element candidates.

The returned Set is *not* Serializable.

If the provided `Set` of `elementCandidates` is subsequently modified, the returned `Set` will not reflect such modifications.

The `Set` of `elementCandidates` must use equals() as its equivalence relation, or its comparison method must be consistent with `equals()`, otherwise the behavior is unspecified.

Here is an example involving an application that manages various configurable options -- commonly referred to as "switches" -- that control its behavior. The state of these switches can be determined through the command line, a configuration file, or even a database connection. By using a lazy set, we ensure that the states of these switches are evaluated only once. Once computed, the results are eligible for constant folding by the JVM:

```
class Application {

    enum Option {VERBOSE, DRY_RUN, STRICT}

    // Lazily initialized Set of Options
    static final Set<Option> OPTIONS =
            Set.ofLazy(EnumSet.allOf(Option.class), Application::isEnabled);

    // Return true when the given Option is enabled
    private static boolean isEnabled(Option option) {
        // Parse command line, read configuration file, load database
        ...
    }

    public static void process() {
        // The if condition (and subsequent eliminated branch) is
        // eligible for constant folding (and code elimination).
        if (OPTIONS.contains(Option.DRY_RUN)) {
            // Skip processing in DRY_RUN mode
            return;
        }
        // Actual processing logic
    }

}
```

The returned `Set<E>` can be thought of as a set backed by a `Map<E, LazyConstant<Boolean>>` field and where the contains(Object) operation is equivalent to:

```
class LazySet<E> extends AbstractCollection<E> implements Set<E> {

    private final Map<E, LazyConstant<Boolean>> backingMap;

    public LazySet(Set<E> elementCandidates, Predicate<E> computingFunction) {
        this.backingMap = elementCandidates.stream()
                .collect(Collectors.toUnmodifiableMap(
                        Function.identity(),
                        k -> LazyConstant.of(() -> computingFunction.test(k))));
    }

    @Override
    public boolean contains(Object o) {
        var lazyConstant = backingMap.get(o);
        return lazyConstant == null
                ? false
                : lazyConstant.get();
    }
}
```

Except, performance and storage efficiency might be better.

Elements in the returned set are eligible for certain performance optimizations such as *constant folding* as described in LazyConstant.

Implementation Note:

after all element membership statuses have been initialized successfully or transitioned to an error state, the computing function is no longer strongly referenced and becomes eligible for garbage collection.

Type Parameters:

`E` - the type of elements maintained by the returned set

Parameters:

`elementCandidates` - the (non-null) element candidates to be evaluated

`computingFunction` - to invoke whenever the membership status of an element candidate is first computed

Returns:

a new lazily computed set whose logical membership for each distinct element candidate in the set of `elementCandidates` is computed via the provided `computingFunction` on demand

Throws:

`NullPointerException` - if the provided set of `elementCandidates` is `null`, if the set of `elementCandidates` contains a `null` element, or if the provided `computingFunction` is `null`

Since:

27

See Also:

`LazyConstant` PREVIEW

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.